

Exception Handling Using Try Catch in SQL Server

In this article, I am going to discuss **Exception Handling Using Try Catch in SQL Server** with examples. Please read our previous article where we discussed [how to raise errors explicitly in SQL Server](#) using RaiseError and Throw statement. As part of this article, we are going to discuss the following pointers.

1. [How to use Try Catch in SQL Server to Handle Error?](#)
2. [Example To understand the Try-Catch implementation in SQL Server.](#)
3. [Try-catch Implementation in SQL Server with system-defined error statements.](#)
4. [What is ERROR_MESSAGE\(\) in SQL Server?](#)
5. [A real-time example of Exception handling using the try-catch in SQL Server.](#)

How to use Try Catch in SQL Server to Handle Error?

From SQL Server 2005 we are provided with a structure error handling mechanism with the help of **TRY** and **CATCH** blocks. The syntax of using **TRY- CATCH** is shown in the below image.

```
BEGIN TRY
    -- statements which will cause the error
    -- statements which are related with the error that should not be execute when the error occurred
END TRY
BEGIN CATCH
    -- statements which should be executed when the error occurs
END CATCH
-- Any Other Statements
```

The **SQL statements** which can have the possibility to throw an exception need to be placed in between the **BEGIN TRY** and **END TRY** blocks. If there is an exception that occurred in the TRY block, then the control immediately moves to the Corresponding **CATCH** block. If there is no exception occurred in the TRY block, then the **CATCH** block simply skipped, and the statements which are present after the **CATCH** block are going to be executed.

Note: The Errors trapped by a CATCH block are not going to be returned to the calling application. If you want to return the error information back to the calling application then you need to use the **RAISERROR()** function explicitly with the catch block. In our previous article, we discussed [how to raise errors explicitly using the RAISERROR\(\) function](#).

Example: To understand the Try-Catch implementation in SQL Server.

In the following example, we create a stored procedure for dividing 2 variables values by using the SQL Server **TRY CATCH** implementation with user-defined error statements.

```
CREATE PROCEDURE spDivideTwoNumbers
@Number1 INT,
@Number2 INT
AS
BEGIN
    DECLARE @Result INT
```

```

SET @Result = 0
BEGIN TRY
    SET @Result = @Number1 / @Number2
    PRINT 'RESULT IS : '+CAST(@Result AS VARCHAR)
END TRY
BEGIN CATCH
    PRINT 'SECOND NUMBER SHOULD NOT BE ZERO'
END CATCH
END

```

EXEC spDivideTwoNumbers 10, 2

OUTPUT: RESULT IS : 5

EXEC spDivideTwoNumbers 10, 0

OUTPUT: SECOND NUMBER SHOULD NOT BE ZERO

When we execute the above-stored procedure with correct values, then the error will not occur in the program. That means after executing all the statements in the try block the control directly jumps to the statements present after the catch block without executing the catch block.

If any error occurs in the execution process i.e. in the try block, then in such case from the line where the error got occurred, the control directly jumps to the catch block. So rest of the statements in the try block will not execute whereas the catch block will execute.

Note: In the above program when the error got occurred, we are displaying an error message “**SECOND NUMBER SHOULD NOT BE ZERO**”. In place of that error message we can also display the original error message by calling a function “**Error_Message**”. To test this rewriting the code inside the catch block as following

Print Error_Message()

Example: try-catch with system defined error statements in SQL Server:

In the following example, we create a stored procedure for dividing two variables values by using try-catch implementation with system-defined error statements in SQL Server.

```

ALTER PROCEDURE spDivideTwoNumbers
@Number1 INT,
@Number2 INT
AS
BEGIN
    DECLARE @Result INT
    SET @Result = 0
    BEGIN TRY
        SET @Result = @Number1 / @Number2
        PRINT 'RESULT IS : '+CAST(@Result AS VARCHAR)
    END TRY
    BEGIN CATCH
        PRINT ERROR_MESSAGE()
    END CATCH
END

```

END

Execution: **EXEC spDivideTwoNumbers 10, 0**

OUTPUT: **Divide by zero error encountered.**

What is ERROR_MESSAGE() in SQL Server?

This method is used to display what type of error has occurred in the try block.

A real-time example of using the try-catch implementation in SQL Server.

We are going to use the following Product and ProductSales table to understand how to handle errors in SQL Server using RaiseError and @ERROR System-Defined Functions.

Product				ProductSales		
ProductId	Name	Price	QuantityAvailable	ProductSalesId	ProductId	QuantitySold
101	Laptop	1234	100	1	101	5
102	Desktop	3456	50	2	102	7
103	Tablet	5678	35	3	103	5
104	Mobile	7890	25	4	104	7

Please use the below SQL Script to create and populate the Product and ProductSales table with sample data.

```
-- Create Product table
CREATE TABLE Product
(
    ProductId INT PRIMARY KEY,
    Name VARCHAR(50),
    Price INT,
    QuantityAvailable INT
)
GO
-- Populate the Product Table with some test data
INSERT INTO Product VALUES(101, 'Laptop', 1234, 100)
INSERT INTO Product VALUES(102, 'Desktop', 3456, 50)
INSERT INTO Product VALUES(103, 'Tablet', 5678, 35)
INSERT INTO Product VALUES(104, 'Mobile', 7890, 25)
GO
-- Create ProductSales table
CREATE TABLE ProductSales
(
    ProductSalesId INT PRIMARY KEY,
    ProductId INT,
```

```

    QuantitySold INT
)
GO
-- Populate the ProductSales Table with some test data
INSERT INTO ProductSales VALUES(1, 101, 5)
INSERT INTO ProductSales VALUES(2, 102, 7)
INSERT INTO ProductSales VALUES(3, 103, 5)
INSERT INTO ProductSales VALUES(4, 104, 7)
Go

```

Stored procedure for product sales using TRY Catch Implementation in SQL Server

The following stored procedure accepts 2 parameters i.e. ProductID and QuantityToSell. The ProductID parameter specifies the product that we want to sell and the QuantityToSell parameter specifies the quantity that we want to sell. In the below procedure, if enough stock is not available then we are raising a custom exception by using the Raiserror statement. If enough stock is available, then we are performing the required operation as part of a transaction and moreover, the transaction is within the Begin TRY and End TRY block.

```

CREATE PROCEDURE spSellProduct
@ProductID INT,
@QuantityToSell INT
AS
BEGIN
    -- First we need to Check the stock available for the product we want to sell
    DECLARE @StockAvailable INT
    SELECT @StockAvailable = QuantityAvailable
    FROM Product
    WHERE ProductId = @ProductId
    -- We need to throw an error to the calling application
    -- if the stock is less than the quantity we want to sell
    IF(@StockAvailable < @QuantityToSell)
    BEGIN
        Raiserror('Enough Stock is not available',16,1)
    END
    -- If enough stock is available
    ELSE
    BEGIN
        BEGIN TRY
            -- We need to start the transaction
            BEGIN TRANSACTION
            -- First we need to reduce the quantity available
            UPDATE Product SET
                QuantityAvailable = (QuantityAvailable - @QuantityToSell)
            WHERE ProductID = @ProductID
            -- Calculate MAX ProductSalesId
            DECLARE @MaxProductSalesId INT
            SELECT @MaxProductSalesId = CASE
                WHEN MAX(ProductSalesId) IS NULL THEN 0
                ELSE MAX(ProductSalesId)
            END
        END TRY
        BEGIN CATCH
            -- Rollback the transaction
            ROLLBACK TRANSACTION
        END CATCH
    END
END

```

```

FROM ProductSales
-- Increment @MaxProductSalesId by 1, so we don't get a primary key viol
Set @MaxProductSalesId = @MaxProductSalesId + 1
-- We need to insert the quantity sold into the ProductSales table
INSERT INTO ProductSales(ProductSalesId, ProductId, QuantitySold)
VALUES(@MaxProductSalesId, @ProductId, @QuantityToSell)
COMMIT TRANSACTION
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION
    SELECT ERROR_NUMBER() as ErrorNumber,
           ERROR_MESSAGE() as ErrorMessage,
           ERROR_PROCEDURE() as ErrorProcedure,
           ERROR_STATE() as ErrorState,
           ERROR_SEVERITY() as ErrorSeverity,
           ERROR_LINE() as ErrorLine
END CATCH
End
END

```

In the procedure spSellProduct the Begin Transaction and Commit Transaction statements are wrapped between the Begin Try and End Try block. If there is no error occurred in the code that is enclosed within the BEGIN TRY and END TRY block, then the COMMIT TRANSACTION statement gets executed and the changes are made permanent to the database.

On the other hand, if there is an error occurred within the try block then the control immediately jumps to the CATCH block, and in the CATCH block, we are rolling back the transaction. So it's much easier to handle errors with the Try/Catch construct than with the @@Error system function in SQL Server.

SQL Server also provides some built-in functions that we can use in the scope of a CATCH block which is used to retrieve more information about the error that occurred and these functions will return NULL if they are executed outside the scope of the CATCH block.

Note: We cannot use the **TRY/CATCH** implementation within a user-defined function.

In the next article, I am going to discuss [Views in SQL Server](#). Here, in this article, I try to explain **Exception Handling Using Try Catch in SQL Server** with Examples. I hope you enjoy this Exception Handling Using Try Catch in SQL Server article and understand how to handle exceptions in SQL Server using the Try-Catch block.